

Report for Project 3

姓名：曾清

学号：524031910374

报告分为三个部分，分别是数独校验、多线程排序 (C)、Fork-Join 并行排序 (Java) 的代码解释。各子任务的测试截图在附录中。

1. Sudoku Validator

整体思路是创建 11 个线程分别完成校验：1 个线程检查所有行，1 个线程检查所有列，9 个线程各自检查一个 3×3 子方格。各线程将校验结果写入全局数组 `results`，主线程汇总判断。

所有校验线程共用同一个辅助函数 `is_valid`，通过 `check[val]` 标记数组检测重复，逻辑简单不再展开。这里重点解释子方格线程的索引计算：

```
int index = 2 + (start_row / 3) * 3 + (start_col / 3);
if (is_valid(grid_data)) results[index] = 1;
free(p);
```

`start_row / 3` 和 `start_col / 3` 分别得到该子方格在 3×3 大格中的行列编号 (0~2)，乘 3 相加后加上偏移量 2，将 9 个子方格一一映射到 `results[2]~results[10]`。参数 `p` 由主线程 `malloc` 传入，在线程内 `free` 以避免内存泄漏。

2. Multithreaded Sort (C)

整体思路是将数组从中间分为两半，用两个线程分别对两半做冒泡排序，等两个线程均完成后，第三个线程再将两段有序子数组归并到全局数组 `sorted` 中。

排序线程操作原数组 `list` 的对应区间，归并线程则读取排好序的两段进行合并。归并线程的参数复用了同一个 `parameters` 结构体，`beg_idx` 存放分界点 `mid`，`end_idx` 存放末尾，核心归并逻辑如下：

```
int k = 0, i = 0, j = mid;
while (i < mid && j < tail) {
    while (i < mid && list[i] <= list[j]) sorted[k++] = list[i++];
    while (j < tail && list[j] <= list[i]) sorted[k++] = list[j++];
}
while (i < mid) sorted[k++] = list[i++];
while (j < tail) sorted[k++] = list[j++];
```

左半段下标从 0 到 `mid-1`，右半段从 `mid` 到 `tail-1`，双指针交替推进，最后将剩余元素追加到 `sorted`。

3. Fork-Join Parallel Sort (Java)

两种排序算法均继承 `RecursiveAction`，通过 `ForkJoinPool` 调度。共同点是设置阈值 `THRESHOLD = 100`：子数组规模足够小时退化为插入排序，否则递归拆分并用 `invokeAll` 并行执行子任务。

QuickSort: `compute` 中先调用 `partition` 以最左元素为基准做双指针原地划分，得到基准落定位置 `pivotIndex` 后，对左右两段各创建一个 `QuickSort` 子任务并行执行：

```
int pivotIndex = partition(arr, left, right);
QuickSort leftTask = new QuickSort(arr, left, pivotIndex - 1);
QuickSort rightTask = new QuickSort(arr, pivotIndex + 1, right);
invokeAll(leftTask, rightTask);
```

MergeSort: `compute` 中对左右两段并行递归，`invokeAll` 返回后（即两段均已有序）再调用 `merge` 原地归并（归并策略和任务二多线程排序中的算法一致，不再赘述），合并结果直接写回 `arr`：

```
invokeAll(leftTask, rightTask);
merge(arr, left, mid, right);
```

与 QuickSort 不同，MergeSort 的合并步骤必须在两个子任务**全部完成后**串行执行，这是 `invokeAll` 阻塞语义的关键用途所在。

Appendix for Test Screenshots

1. test for sudoku:

```
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sudoku$ cat valid.txt
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sudoku$ ./sudoku < valid.txt
Sudoku is VALID!
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sudoku$ cat invalid.txt
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 5
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sudoku$ ./sudoku < invalid.txt
Sudoku is INVALID!
```

2. test for multithread-sorting, MergeSort and QuickSort:

```
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sudoku$ cd ../sort
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sort$ ./sort
2 6 4 3 7 34 78 43 5 7 12 90
2 3 4 5 6 7 7 12 34 43 78 90
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/sort$ cd ../ForkJoin/
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/ForkJoin$ java MergeSort
5 2 4 78 4 32 8 564 96 879 55 3
The sorted data:
2 3 4 4 5 8 32 55 78 96 564 879
wynter@wynter-VMware-Virtual-Platform:~/os_projects/project_3/ForkJoin$ java QuickSort
56 8 5 34 768 3 0 6 45 87 3 9 72 47 97 54
The sorted data:
0 3 3 5 6 8 9 34 45 47 54 56 72 87 97 768
```